

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 6

Operating Systems

Topic: Inside the xv6 File System

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Purpose of the Work 3

2 Part 1 – Read What xv6 Shows You 3

2.1 Observations 5

3 Part 2 – Write `mystat` 5

3.1 Observations 7

4 Part 3 – Measure the Limits 8

4.1 Observations 8

5 Part 4 – Write `find` 9

6 Conclusion 11

1. Purpose of the Work

The purpose of this laboratory work is to look inside the xv6 file system and connect the concepts from the previous Linux laboratory with their simpler xv6 implementation. I inspected file types and inode numbers, wrote my own `mystat` utility, measured limits defined in `kernel/fs.h`, and implemented a recursive `find` command that walks through directories.

2. Part 1 – Read What xv6 Shows You

I started by looking at the normal `ls` output in xv6, then created an empty directory and inspected what xv6 stores inside it.

Output

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
sleep          2 5 35280
forktest       2 6 18136
args           2 7 35272
grep           2 8 44184
init           2 9 35976
kill           2 10 35512
ln             2 11 35304
ls             2 12 42880
mkdir          2 13 35568
rm             2 14 35552
sh             2 15 58312
stressfs       2 16 36416
usertests      2 17 206800
grind          2 18 51824
wc             2 19 37728
zombie         2 20 34848
logstress      2 21 37560
forphan        2 22 36352
dorphan        2 23 35800
sync           2 24 34944
console        3 25 0
$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
```

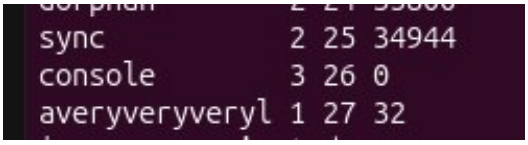


Figure 1.1 – The end of the xv6 root directory listing.

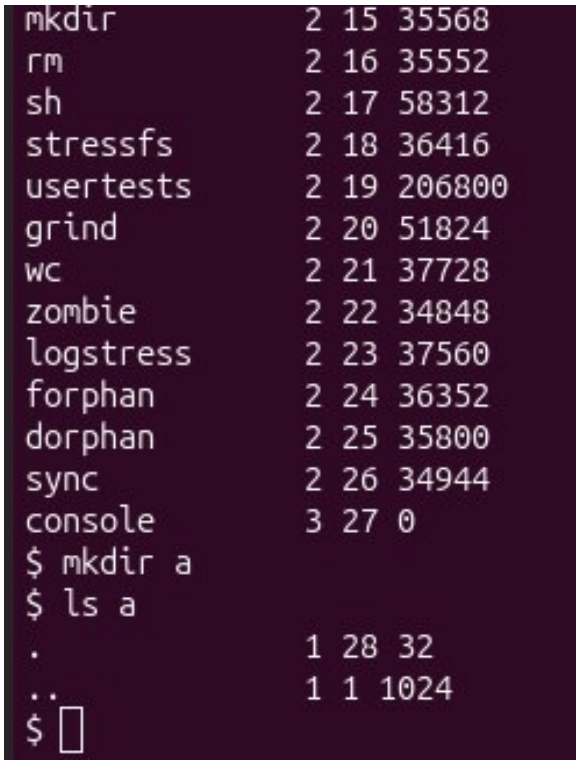


Figure 1.2 – Creating directory `a` and listing its entries.

2.1. Observations

1. What do types 1, 2 and 3 mean?

I checked `kernel/stat.h` and got:

kernel/stat.h

```
#define T_DIR    1 // Directory
#define T_FILE   2 // File
#define T_DEVICE 3 // Device
```

So thats what 1, 2 and 3 means: type 1 is a directory, type 2 is a normal file, and type 3 is a device.

2. Which inode number does the root directory have, and why are `.` and `..` the same there?

Both `.` and `..` have the same inode nr (1) because they both point to the main inode, which is the main folder. Since the root directory has no parent above it, its parent entry points back to root itself.

3. How big is one directory entry, and which entries does an empty directory contain?

A entry in the directory has a fixed size of 16 bytes. The 2 entries contained in the folder are `.` (current dir) and `..` (parent dir), which is why an empty directory has a size of 32 bytes.

3. Part 2 – Write `mystat`

For this part I implemented my own small version of `stat`. It calls `stat()` for every argument, translates the type number into a readable name, and prints the inode number, link count and size.

user/mystat.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  // DONE: return "directory", "file" or "device" for T_DIR, T_FILE, T_DEVICE
  switch(t)
  {
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
```

```
  case T_DEVICE:
    return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    // DONE: call stat(argv[i], &st); on error print to fd 2 and continue
    if (stat(argv[i], &st) == -1)
    {
      fprintf(2, "Unable to inrerpret %s\n", argv[i]);
      continue;
    }
    // DONE: print NAME: TYPE, inode N, links N, size N bytes
    printf("%s: ", argv[i]); //written on my fucking own
    printf("%s, ", typename(st.type));
    printf("inode %u, ", st.ino);
    printf("links %u, ", st.nlink);
    printf("size %lu bytes\n", st.size);
  }
  exit(0);
}
```

I then used the program on a regular file, the current directory, and the console device, followed by the hard-link experiment.

Output

```
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

```
findredhundred@Ubuntu:~/xv6-riscv/kernel$ cd ..
findredhundred@Ubuntu:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-leg
,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mystat README . console
Unable to inrerpret README
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
$
```

Figure 2.1 – Running `mystat` and checking the hard-linked file after removing the original name.

3.1. Observations

1. What changed after `ln`, and what changed after `rm`?

Links has been incremented. Nothing else has changed. They simply point to the same inode. After creating `r2`, both names showed inode 2 and link count 2. After removing `README`, `r2` still had inode 2 and the same size, while the link count returned to 1. The data was still readable through `r2`.

2. Where is the link count decremented, and when are the blocks freed?

In my notes I identified the decrement in this section of `sys_unlink`:

Comment from my notes

```
memset(&de, 0, sizeof(de));
if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
  panic("unlink: writei");
if (ip->type == T_DIR) {
  dp->nlink--; // <------------- HERE
  iupdate(dp);
}
```

The important idea from the experiment is that removing one name does not immediately remove the file data while another hard link still exists. The notes do not include the later block-freeing code, but the blocks are only released once the inode no longer has links keeping the file reachable and the inode can be freed.

3. Which fields does an xv6 on-disk inode contain?

These are the fields I found in `struct dinode`:

kernel/fs.h

```
struct dinode {
  short type;              // File type
  short major;             // Major device number (T_DEVICE only)
  short minor;             // Minor device number (T_DEVICE only)
```

```
  short nlink;             // Number of links to inode in file system
  uint size;               // Size of file (bytes)
  uint addrs[NDIRECT + 1]; // Data block addresses
};
```

These are the fields.

4. Part 3 – Measure the Limits

I inspected the constants that define the xv6 file-system limits and rebuilt the file-system image to see the disk layout printed by `mkfs`.

Commands and output

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ"
   kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14

$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks
     1953 total 2000
```

```
findredhundred@Ubuntu:~/xv6-riscv/kernel$ cat fs.h
// On-disk file system format.
// Both the kernel and user programs use this header file.

#define ROOTINO 1    // root i-number
#define BSIZE   1024 // block size

// Disk layout:
// [ boot block | super block | log | inode blocks |
//                                          free bit map | data blocks]
//
// mkfs computes the super block and builds an initial file system. The
// super block describes the disk layout:
struct superblock {
```

Figure 3.1 – File-system constants and the disk-layout line printed while building `fs.img`.

I also opened `kernel/fs.h` to connect these constants with the documented xv6 disk layout.

4.1. Observations

1. What does xv6 show for a name longer than `DIRSIZ`, and why?

It showed `averyveryveryl`. My comment was that this happens because the directory entry is defined using 2 bytes for the inode number and 14 chars for the name. In other words,

```
findredhundred@Ubuntu:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
findredhundred@Ubuntu:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_sleep user/_forktest user/_args user/_grep user/_init user/_kill user/_mystat user/_ln user/_l
sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_syn
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

Figure 3.2 – The disk-layout description in `kernel/fs.h`.

`DIRSIZ` is 14, so the longer name is limited to that directory-entry name size.

2. What is the largest file xv6 can hold?

12 `NDIRECT` + 1 `INDIRECT`, therefore

$$(12 + \texttt{NINDIRECT}) \times \texttt{BSIZE} = \left(12 + \frac{1024}{4}\right) \times 1024 = 274432 \text{ bytes}$$

which is approximately 268 KiB, so roughly 270 KB as I wrote in my notes.

3. How is the 2000-block disk divided?

The whole disk has a total of 2k blocks, of which 31 are for the log, 13 for inode blocks and 1953 for data. The output also reports the boot block, superblock and one bitmap block as part of the 47 metadata blocks.

5. Part 4 – Write `find`

For the final part I wrote a recursive `find` command. It opens a directory, reads every `dirent`, skips unused entries and the special `.` and `..` entries, builds a full path, prints matching names, and recursively enters subdirectories.

user/find.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, 0)) < 0) {
```

```
  fprintf(2, "find: cannot open %s\n", path);
  return;
}

if (fstat(fd, &st) < 0) {
  fprintf(2, "find: cannot stat %s\n", path);
  close(fd);
  return;
}

if (st.type != T_DIR) {
  close(fd);
  return;
}

if (strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
  fprintf(2, "find: path too long\n");
  close(fd);
  return;
}

strcpy(buf, path);
p = buf + strlen(buf);
*p++ = '/';

while (read(fd, &de, sizeof(de)) == sizeof(de)) {
  if (de.inum == 0)
    continue;

  if (strcmp(de.name, ".") == 0 ||
      strcmp(de.name, "..") == 0)
    continue;

  memmove(p, de.name, DIRSIZ);
  p[DIRSIZ] = 0;

  if (stat(buf, &st) < 0)
    continue;

  if (strcmp(de.name, name) == 0)
    printf("%s\n", buf);

  if (st.type == T_DIR)
    find(buf, name);
}

close(fd);
```

```
}

int
main(int argc, char *argv[])
{
  if (argc != 3) {
    fprintf(2, "Usage: find <dir> <name>\n");
    exit(1);
  }

  find(argv[1], argv[2]);
  exit(0);
}
```

I tested it with nested directories and several files named `r2`.

Test

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

The output shows that the program correctly entered `a`, then `a/b`, and printed every path whose last component matched `r2`. Running `find a b` also correctly found the nested directory `a/b`.

6. Conclusion

This laboratory made the xv6 file system much easier to understand because I could connect the source definitions directly to visible behavior. I saw how xv6 represents file types and directory entries, how hard links are only different names for the same inode, and how the link count changes when names are added or removed. Writing `mystat` made the inode information visible, while checking `fs.h` showed that even limits such as file-name length and maximum file size come directly from simple constants. The final `find` program tied everything together by reading directory entries and recursively walking the file-system tree.

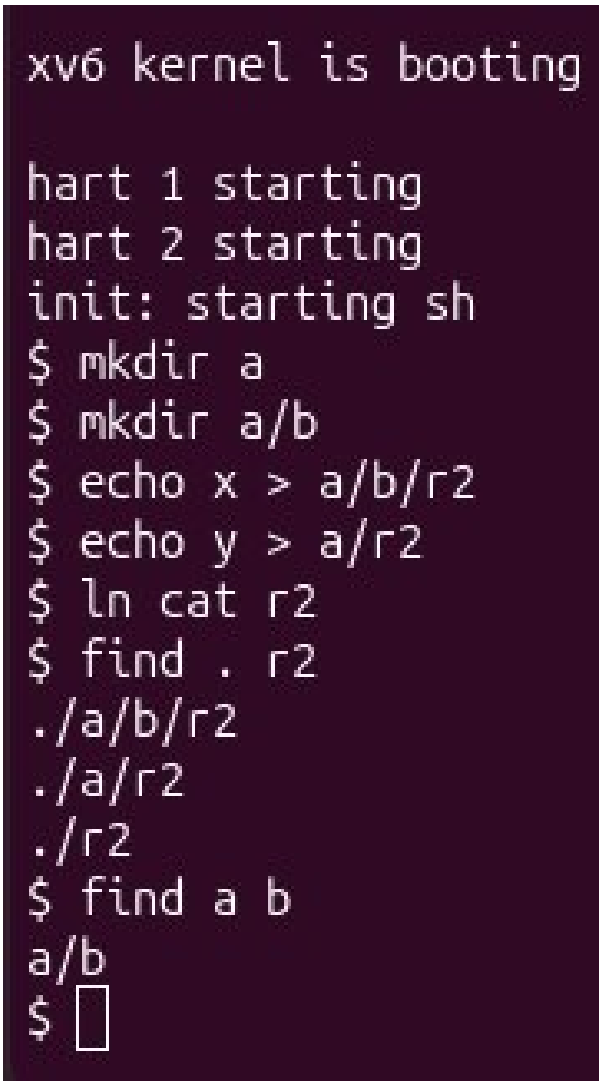


Figure 4.1 – Testing the recursive `find` command in xv6.